```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m33.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m33.c -o m33
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m33
==========================================
       COMBINED OUTPUT REDIRECTION
==========================================
Enter output filename: NIece

stdout and stderr were merged.
Output file: NIece
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_FILENAME 256

int main()
{
    char filename[MAX_FILENAME];
    int fd;
    pid_t pid;
    int status;

    printf("==========================================\n");
    printf("       COMBINED OUTPUT REDIRECTION\n");
    printf("==========================================\n");

    printf("Enter output filename: ");

    if (fgets(filename, sizeof(filename), stdin) == NULL)
        return 1;

    filename[strcspn(filename, "\n")] = '\0';

    fd = open(
        filename,
        O_WRONLY | O_CREAT | O_TRUNC,
        0644
    );

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        close(fd);
```

```
    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        close(fd);
        return 1;
    }

    if (pid == 0)
    {
        /*
         * Redirect stdout to file
         */
        if (dup2(fd, STDOUT_FILENO) == -1)
        {
            perror("dup2 stdout");
            exit(EXIT_FAILURE);
        }

        /*
         * Redirect stderr to the SAME file
         */
        if (dup2(fd, STDERR_FILENO) == -1)
        {
            perror("dup2 stderr");
            exit(EXIT_FAILURE);
        }

        close(fd);

        /*
         * stdout output
```

```
        close(fd);

        /*
         * stdout output
         */
        printf("This is standard output.\n");
        fflush(stdout);

        /*
         * stderr output
         */
        fprintf(stderr,
                "This is standard error.\n");

        /*
         * Generate command output and error
         */
        execlp(
            "ls",
            "ls",
            "-l",
            "existing_file.txt",
            "missing_file.txt",
            NULL
        );

        perror("execlp");
        exit(EXIT_FAILURE);
    }

    close(fd);

    waitpid(pid, &status, 0);

    printf("\nstdout and stderr were merged.\n");
    printf("Output file: %s\n", filename);

    return 0;
}
```

```
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ nano m34.c
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ gcc m34.c -o m34
anirudh@LAPTOP-H9V5A2PN:~/OSLAB$ ./m34
==========================================
   COMPLEX PIPE AND REDIRECTION SHELL
==========================================

Examples:
cat < input.txt | grep Linux > output.txt
cat < input.txt | sort | uniq > output.txt
cat input.txt | grep hello >> result.txt
ps aux | grep bash | wc -l
exit

complex-shell> ps

==========================================
             EXECUTION PLAN
==========================================

Command 1:
Program: ps
Arguments: ps

Executing...
    PID TTY          TIME CMD
    377 pts/0    00:00:00 bash
   1132 pts/0    00:00:00 m34
   1135 pts/0    00:00:00 ps

Execution completed.

complex-shell> ^C
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>

#define MAX_INPUT 1024
#define MAX_COMMANDS 10
#define MAX_ARGS 20
#define MAX_FILENAME 256

typedef struct
{
    char *args[MAX_ARGS];
    int argc;

    char input_file[MAX_FILENAME];
    char output_file[MAX_FILENAME];

    int append;
} Command;

int parse_command(char *text, Command *cmd)
{
    char *token;
    int argc = 0;

    cmd->input_file[0] = '\0';
    cmd->output_file[0] = '\0';
    cmd->append = 0;

    token = strtok(text, " \t\n");

    while (token != NULL)
    {
        if (strcmp(token, "<") == 0)
        {
            token = strtok(NULL, " \t\n");

            if (token == NULL)
            {
                printf("Syntax error after <\n");
                return 0;
            }

            strcpy(cmd->input_file, token);
        }
        else if (strcmp(token, ">") == 0)
        {
            token = strtok(NULL, " \t\n");

            if (token == NULL)
            {
                printf("Syntax error after >\n");
                return 0;
            }

            strcpy(cmd->output_file, token);
            cmd->append = 0;
        }
        else if (strcmp(token, ">>") == 0)
        {
            token = strtok(NULL, " \t\n");

            if (token == NULL)
            {
                printf("Syntax error after >>\n");
                return 0;
            }

            strcpy(cmd->output_file, token);
            cmd->append = 1;
        }
        else
        {
```

```
        else
        {
            if (argc >= MAX_ARGS - 1)
            {
                printf("Too many arguments.\n");
                return 0;
            }

            cmd->args[argc++] = token;
        }

        token = strtok(NULL, " \t\n");
    }

    cmd->args[argc] = NULL;
    cmd->argc = argc;

    return argc > 0;
}

int parse_pipeline(
    char *input,
    Command commands[])
{
    char *parts[MAX_COMMANDS];
    int count = 0;

    char *start = input;
    char *pipe_position;

    while ((pipe_position = strchr(start, '|')) != NULL)
    {
        if (count >= MAX_COMMANDS - 1)
        {
            printf("Too many commands.\n");
            return -1;
        }

        *pipe_position = '\0';

        parts[count++] = start;

        start = pipe_position + 1;
    }

    parts[count++] = start;

    for (int i = 0; i < count; i++)
    {
        if (!parse_command(parts[i],
                           &commands[i]))
        {
            return -1;
        }
    }

    return count;
}

void display_plan(
    Command commands[],
    int count)
{
    int i;
    int j;

    printf("\n==========================================\n");
    printf("             EXECUTION PLAN\n");
    printf("==========================================\n");

    for (i = 0; i < count; i++)
    {
        printf("\nCommand %d:\n", i + 1);

        printf("Program: %s\n",
               commands[i].args[0]);
```

```
            );
        }

        if (i < count - 1)
        {
            printf("Output connected to command %d\n",
                   i + 2);
        }
    }
}

void execute_pipeline(
    Command commands[],
    int count)
{
    int pipes[MAX_COMMANDS - 1][2];
    pid_t pids[MAX_COMMANDS];

    int i;
    int j;

    for (i = 0; i < count - 1; i++)
    {
        if (pipe(pipes[i]) == -1)
        {
            perror("pipe");
            exit(EXIT_FAILURE);
        }
    }

    for (i = 0; i < count; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            exit(EXIT_FAILURE);
        }

        if (pids[i] == 0)
        {
            /*
             * Pipeline input
             */
            if (i > 0)
            {
                if (dup2(
                        pipes[i - 1][0],
                        STDIN_FILENO) == -1)
                {
                    perror("dup2");
                    exit(EXIT_FAILURE);
                }
            }

            /*
             * Pipeline output
             */
            if (i < count - 1)
            {
```

```
     * Parent waits for children
     */
    for (i = 0;
         i < count;
         i++)
    {
        waitpid(
            pids[i],
            NULL,
            0
        );
    }
}

int main()
{
    char input[MAX_INPUT];

    Command commands[MAX_COMMANDS];

    int count;

    printf("============================================\n");
    printf("   COMPLEX PIPE AND REDIRECTION SHELL\n");
    printf("============================================\n");

    printf("\nExamples:\n");
    printf("cat < input.txt | grep Linux > output.txt\n");
    printf("cat < input.txt | sort | uniq > output.txt\n");
    printf("cat input.txt | grep hello >> result.txt\n");
    printf("ps aux | grep bash | wc -l\n");
    printf("exit\n");

    while (1)
    {
        printf("\ncomplex-shell> ");
        fflush(stdout);

        if (fgets(input,
                  sizeof(input),
                  stdin) == NULL)
        {
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        count =
            parse_pipeline(
                input,
                commands
            );

        if (count <= 0)
        {
            printf("Invalid command syntax.\n");
```

```
                input,
                commands
            );

        if (count <= 0)
        {
            printf("Invalid command syntax.\n");
            continue;
        }

        display_plan(
            commands,
            count
        );

        printf("\nExecuting...\n");

        execute_pipeline(
            commands,
            count
        );

        printf("\nExecution completed.\n");
    }

    return 0;
}
```